Programming Fundamentals

Project Report

Objective: Climate Change Impact Calculator

Submitted by:

Leader Name: Wasif Ali Reg. No: 245187

Member Name: Anisa Khan Reg. No: 245215

Member Name: Jawad Mustafa Reg. No: 245195

Member Name: Faiqa Umar Reg. No: 245161

Member Name: Mubashir Ali Reg. No: 245185

Submitted to:

SIR GHULAM ALI



Department of Computer Science

Air University - Aerospace and Aviation Campus,Kamra

Table of Contents

CHAPTER 1: PROBLEM STATEMENT 4
1.1 Statement 4
1.2 Background 4
1.3 Objective 4
1.4 Significance 4
CHAPTER 2: INTRODUCTION 5
2.1 Overview 5
2.2 Motivation 5
2.3 Relevance 5
CHAPTER 3: ALGORITHM AND FLOWCHART 5
3.1 Algorithm 5
3.2 Pseudocode 7
3.3 Flowchart 10
CHAPTER 4: DESIGN AND IMPLEMENTATION 11
4.1 System Design 11
4.2 Modules 13
4.3 Data Structures 13
4.4 Code Implementation 13
CHAPTER 5: TESTING 15
5.1 Test Plan 15
5.2 Test Cases 16
5.3 Results 21
5.4 Error Handling 22
CHAPTER 6: USER INTERACTION FEATURES 22
6.1 Menu Option 22
6.2 File Saving 26
CHAPTER 7: ENVIRONMENTAL INSIGHTS 26
7.1 Carbon Offset Calculation 27
7.2 Comparison Insights 28
CHAPTER 8. CHALLENGES AND LIMITATIONS 28
8.1 Challenges 28
8.2 Limitations 32
CHAPTER 9: CONCLUSION 33

9.1 Summary 33
9.2 Future Work 33
CHAPTER 10. APPENDIX **34**
10.1 Full Program Code 34
10.2 Sample Inputs and Outputs 36
10.3 Formulas and Constants 39

CHAPTER 1: PROBLEM STATEMENT

1.1 Statement

Develop a **Climate Change Impact Calculator** program that estimates the carbon footprint of various human activities. The program will calculate the total CO2 emissions based on user input, provide an Air Quality Index (AQI), suggest methods to offset emissions, and deliver actionable environmental tips. Additionally, the program will:

- Offer detailed reports for individual activities.
- Compare different transportation modes to highlight emission differences.
- Predict future emissions based on current activity levels.
- Save results to a file for future reference.

1.2 Background

Climate change is a defining challenge of our time, with carbon emissions playing a critical role in environmental degradation. The need for accessible tools that educate individuals on their contributions to global emissions has never been greater. This project addresses this gap by offering a tailored solution that combines simplicity, accuracy, and practicality.

1.3 Objective

This program is designed to:

- Accurately calculate carbon footprints from user-input data.
- Provide comparative insights into the environmental impact of different activities, such as transportation and electricity usage.
- Suggest actionable recommendations for reducing emissions and improving sustainability.
- Incorporate predictive analytics to estimate future trends.
- Offer file-saving and report generation for easy reference.

1.4 Significance

The significance of this program lies in its ability to convert data into actionable insights. It encourages individuals to take responsibility for their carbon emissions, fostering a culture of environmental awareness and sustainability. Furthermore, it aligns individual actions with global goals for combating climate change.

CHAPTER 2: INTRODUCTION

2.1 Overview

In an era of increasing environmental concerns, there is a growing need for tools that allow individuals to quantify their carbon footprint and understand the impact of their daily activities on the environment. This program aims to provide a user-friendly solution to calculate and analyze carbon emissions across various activities (e.g., travel, electricity usage) and suggest actionable measures to reduce environmental impact. Additionally, it offers personalized insights such as the Air Quality Index (AQI), carbon offset strategies, and future emission predictions. The results are visualized via a bar chart and saved for further reference.

2.2 Motivation

The motivation behind this project stems from the urgent need to address climate change at the individual level. By providing a tool that combines accessibility and accuracy, this project empowers users to become active participants in mitigating environmental harm. It bridges the gap between awareness and action, enabling users to align their choices with sustainability goals.

2.3 Relevance

This program is relevant in today's context as it equips users with the knowledge to make environmentally responsible choices. Whether for personal use, corporate accountability, or educational purposes, the program offers valuable insights that align with global efforts to combat climate change.

CHAPTER 3: ALGORITHM AND FLOWCHART

3.1 Algorithm

The Climate Change Impact Calculator follows a structured algorithm

Step 1: ***Start***

Step 2: ***Define Constants and Arrays:***

- *Set the total number of activities as 5.*
- *Create an array of activity names (e.g., "Car travel", "Electricity usage", etc.).*
- *Create an array of predefined CO2 emissions for each activity.*

- *Create an empty array for user inputs.*

Step 3: ***Display Instructions:***

- *Show a welcome message and instructions for entering activity data.*

Step 4: ***Input User Data:***

- *For each activity, ask the user to input the number of units (e.g., kilometers, hours, etc.).*
- *Check if the input is valid (numeric); if not, show an error and stop the program.*

Step 5: ***Calculate Total Emissions:***

- *Multiply each activity's units by its CO2 emission factor and sum them up.*

Step 6: ***Determine Air Quality Index (AQI):***

- *Use the total emissions to calculate AQI based on predefined ranges.*

Step 7: ***Calculate Climate Impact:***

- *Compute a "climate change impact factor" using a logarithmic formula.*

Step 8: ***Estimate Offsets Needed:***

- *Calculate how many trees or solar panels are needed to offset the emissions.*

Step 9: ***Show Results:***

- *Display activity-wise emissions, total emissions, AQI, climate impact factor, and offset suggestions (trees and solar panels).*

Step 10: ***Display Bar Chart:***

- *For each activity, calculate its proportional emissions and display a bar chart.*

Step 11: ***Show Menu for Further Options:***

- *Present a menu with the following options:*
- *View detailed climate report.*
- *Compare car vs. bus emissions.*
- *Get carbon offset suggestions.*
- *Learn environmental tips.*
- *View all options.*
- *Predict future emissions.*

Step 12: ***Handle User Choice:***

- *Based on the selected option, perform the corresponding action and display the results.*
- *Repeat until the user decides to exit.*

Step 13: ***Save Report to File:***

- *Save the climate impact report and other results in a text file.*

Step 14: ***Exit Program:***

- *End the program after saving or user exits.*

Step 15: ***Stop***

3.2 Pseudocode

Here's the **Pseudocode** for the Climate Change Impact Calculator

1. *START*

2*. SET total_activities = 5*

SET activity_names = ["Car travel", "Electricity usage", ...] // Add other activities

SET emission factors = [CO2_factor1, CO2_factor2, ...] //Predefined CO2 emissions per unit

INITIALIZE user_inputs = [] // Empty array to store user inputs

INITIALIZE total_emissions = 0

3. *DISPLAY "Welcome to the Carbon Emission Calculator!"*

DISPLAY "Enter the number of units for each activity to calculate emissions."

4. FOR *i FROM 0 TO (total_activities – 1)*

DO

DISPLAY "Enter the number of units for " + activity_names[i] + ":"

INPUT units

IF *units IS NOT NUMERIC THEN*

DISPLAY "Invalid input. Please restart the program and enter numeric values."

EXIT

END IF

APPEND units TO user_inputs

END FOR

***5. FOR** i FROM 0 TO (total_activities – 1)*

DO

*total_emissions = total_emissions + (user_inputs[i] * emission factors[i])*

END FOR

***6.** SET AQI = CALCULATE_AQI(total_emissions)*

***7.** SET climate_impact = LOGARITHMIC_FORMULA(total_emissions)*

***8.** SET trees_needed = CALCULATE_TREES(total_emissions)*

SET solar_panels_needed = CALCULATE_SOLAR_PANELS(total_emissions)

***9.** DISPLAY "Activity-wise Emissions:"*

***FOR** i FROM 0 TO (total_activities – 1) DO*

*DISPLAY activity_names[i] + ": " + (user_inputs[i] * emission_factors[i]) + " kg CO2"*

END FOR

DISPLAY "Total Emissions: " + total_emissions + " kg CO2"

DISPLAY "Air Quality Index (AQI): " + AQI

DISPLAY "Climate Impact Factor: " + climate_impact

DISPLAY "Offsets Needed: " + trees_needed + " trees or " + solar_panels_needed + " solar panels"

***10.** DISPLAY_BAR_CHART (activity_names, user_inputs, emission_factors)*

***11.** WHILE TRUE*

DO

DISPLAY "Menu Options:"

DISPLAY "1. View Detailed Climate Report"

DISPLAY "2. Compare Car vs. Bus Emissions"

DISPLAY "3. Get Carbon Offset Suggestions"

DISPLAY "4. Learn Environmental Tips"

DISPLAY "5. Predict Future Emissions"

DISPLAY "6. Exit"

DISPLAY "Enter your choice:"

INPUT choice

12.

IF *choice == 1 THEN*

DISPLAY_DETAILED_REPORT()

ELSE IF *choice == 2 THEN*

COMPARE_CAR_BUS_EMISSIONS()

ELSE IF *choice == 3 THEN*

DISPLAY_OFFSET_SUGGESTIONS()

ELSE IF *choice == 4 THEN*

DISPLAY_ENVIRONMENTAL_TIPS()

ELSE IF *choice == 5 THEN*

PREDICT_FUTURE_EMISSIONS()

ELSE IF *choice == 6 THEN*

BREAK

ELSE

DISPLAY "Invalid choice. Please try again."

END IF

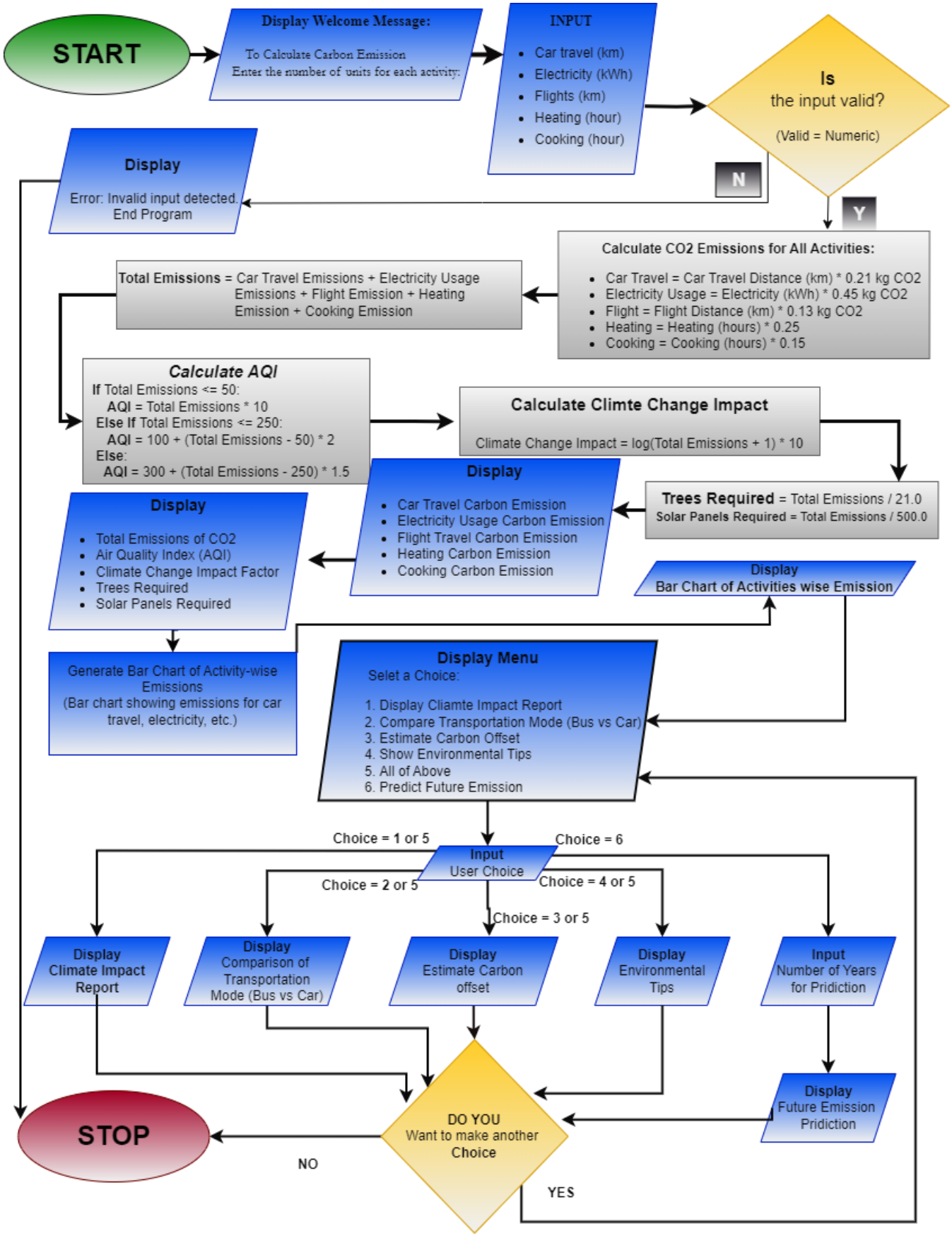
END WHILE

13. *SAVE_TO_FILE ("climate_impact_report.txt", detailed_report)*

14. *DISPLAY "Thank you for using the Carbon Emission Calculator! Goodbye."*

15*. STOP.*

3.3 Flowchart



CHAPTER 4: DESIGN AND IMPLEMENTATION

4.1 System Design

The **Climate Change Impact Calculator** is designed using a modular approach to ensure clarity, scalability, and maintainability. The primary components of the program are:

1. **Input Module**:

- Gathers user input for different activities such as car travel, electricity usage, flights, heating, and cooking.
- Implements robust error handling to ensure only valid numeric inputs are accepted, as shown below:

```
while (true)
  {
    cout << "------------------\n";
    cout << Activities[i] << ": ";
    cin >> User_Input[i];

    if (cin.fail())         // Check for invalid input
    {
        cout << "\nAnalysing Input...\n[";
        for (int i = 0; i < 50; ++i)
        {
            cout << "=" << flush;              // Print the progress bar incrementally
            this_thread::sleep_for(chrono::milliseconds(20)); // Adjust speed with milliseconds
        }
        cout << "] Done!\n";
        cin.clear();        // Clear the error flags
        cin.ignore(100,'\n');   // Ignore invalid input upto 100 character or until new line
        cerr << "\n\aError: Invalid input detected. Please enter numeric values only.\n";
    }
    else if (User_Input[i] < 0)        // Check for negative input
    {
        cout << "\nAnalysing Input...\n[";
        for (int i = 0; i < 50; ++i)
        {
            cout << "=" << flush;              // Print the progress bar incrementally
            this_thread::sleep_for(chrono::milliseconds(20)); // Adjust speed with milliseconds
        }
        cout << "] Done!\n";
        cerr << "\n\aError: Input cannot be negative. Please enter a valid value.\n";
    }
    else
    {
        break;
    }
```

- This ensures the program handles invalid inputs gracefully.

2. **Processing Module**:

- Calculates total emissions for each activity using predefined CO2 emission factors:

```
Total_Emissions += User_Input[i] * CO2_Emission[i];
```

- Determines the Air Quality Index (AQI) using a standardized scale with varying ranges:

```
if (Total_Emissions <= 50)                                          //Standardized Scale:
{                                                                   // 0 to 50   : Good
    AQI = Total_Emissions * 10;  // Linear scaling for low emissions // 51 to 100 : Moderate
}                                                                   // 101 to 150: Unhealthy for sensitive groups
else if (Total_Emissions > 50 && Total_Emissions <= 250)             // 151 to 200: Unhealthy
{                                                                   // 201 to 300: Very Unhealthy
    AQI = 100 + (Total_Emissions - 50) * 2; // Moderate scaling      // 301 to 500: Hazardous
}
else if (Total_Emissions > 250 && Total_Emissions <= 1000)
{
    AQI = 300 + (Total_Emissions - 250) * 1.5; // Higher scaling
}
else
{
    AQI = 500;
}
```

- Calculates carbon offset requirements for trees and solar panels using well-defined formulas:

```
double Trees_Required = Total_Emissions / 21.0; // 21 kg CO2 offset per tree annually
double SolarPanels_Required = Total_Emissions / 500.0; // 1 solar panel offsets 500 kg CO2/year
```

3. **Visualization Module**:

- Provides a bar chart for activity-wise emissions:

```
const int Bar_Scale = 50;  // Adjust the scale of the bar chart
for (int i = 0; i < Total_Activities; i++)
{
    double Activity_Emissions = User_Input[i] * CO2_Emission[i];

    int Bar_Length = (Activity_Emissions * Bar_Scale) / Total_Emissions;   // bar length

    cout << Activities[i] << " [" << Activity_Emissions << " kg CO2]\t: ";

    for (int j = 0; j < Bar_Length; j++)  // Add the bar
    {
        cout << "#";
    }
    cout << "\n";
}
```

- This helps users visualize which activities contribute the most to emissions.

4. **File Management Module**:

- Saves the results and bar chart to a text file for future reference

```
outputFile << Activities[i] << " [" << Activity_Emissions << " kg CO2]: ";
for (int j = 0; j < Bar_Length; j++)
{
    outputFile << "#";
}
outputFile << "\n";
```

4.2 Modules

The program is divided into the following functional modules:

1. **Main Menu**: Provides options to display reports, compare transportation modes, suggest offsets, give tips, and predict future emissions.
2. **Calculation**: Performs all CO2-related computations using constants and user input.
3. **Comparison**: Compares transportation modes and shows potential emission savings.
4. **Visualization**: Outputs activity-specific emissions and bar charts.
5. **File Saving**: Ensures all data and insights are recorded for later review.

4.3 Data Structures

The program uses arrays for efficient storage and retrieval of activity names, user inputs, and CO2 emission factors:

```
string Activities[Total_Activities] = {"Car travel (km)\t\t", "Electricity usage (kWh)\t", "Flights (km)\t\t",
                                       "Heating (hour)\t\t", "Cooking (hour)\t\t"}; // Activities Names Predefined Array
double CO2_Emission[Total_Activities] = {0.21, 0.45, 0.13, 0.25, 0.15};      // Carbon emission Predefined Array
double User_Input[Total_Activities] = {0};
```

4.4 Code Implementation

The program is written in **C++** for efficient execution, incorporating error handling, modular design, and structured programming principles. Libraries such as <iostream>, <fstream>, <thread>, <chrono> and <cmath> are utilized for input/output operations, file handling, and advanced calculations like logarithms for the Climate Change Impact Factor:

```
#include <iostream>
#include <fstream>
#include <thread>      // For sleep functionality
#include <chrono>      // For defining sleep duration
#include <cmath>
```

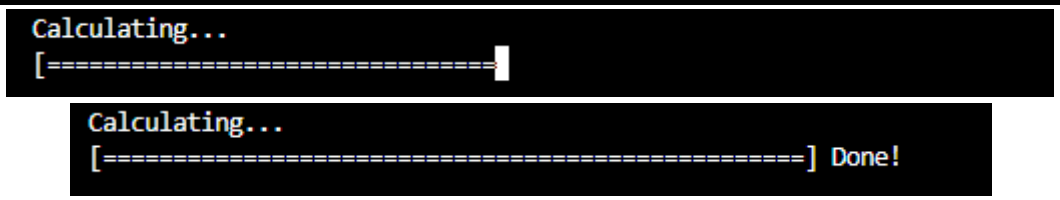
```
double Climate_Change_Impact = log(Total_Emissions + 1) * 10; // Calculate Climate Change Impact Factor
```

The program utilizes the <thread> and <chrono> headers to implement time delays and simulate progress bars. These features enhance the user experience by providing feedback during calculations and report generation. The <thread> header offers functionality for threading and precise control over execution delays, while <chrono> provides utilities for handling time-related functions such as measuring durations and defining sleep intervals.

Advantages:

- Adds realism to the program by simulating real-time processing.
- Improves user engagement and feedback through progress visualization.
- Provides precise control over time-based operations.

```
cout << "Calculating...\n[";
for (int i = 0; i < 50; ++i)
{
    cout << "=" << flush;              // Print the progress bar incrementally
    this_thread::sleep_for(chrono::milliseconds(30)); // Adjust speed with milliseconds
}
cout << "] Done!\n";
```



4.4 Bar Chart Visualization

The program includes a bar chart feature to provide a clear and proportional visualization of emissions from various activities. This enhances the user's ability to identify high-emission activities and make informed decisions to reduce their carbon footprint.

Implementation Details:

- **Purpose:**
  - Helps users understand the contribution of each activity to their total emissions.
  - Highlights which activities have the highest environmental impact.
- **Calculation Method:**
  - Each activity's emissions are calculated as:

$$\text{Activity Emissions} = \text{User Input} \times \text{CO2 Emission Factor}$$

- The length of each bar is proportional to the activity's emissions relative to the total emissions:
  - $Bar\ Length = \frac{Activity\ Emissions \times Bar\ Scale}{Total\ Emission}$

➢ **Bar Scale:** A constant used to adjust bar length for readability.

```
const int Bar_Scale = 50;  // Adjust the scale of the bar chart
for (int i = 0; i < Total_Activities; i++)
{
    double Activity_Emissions = User_Input[i] * CO2_Emission[i];
    int Bar_Length = (Activity_Emissions * Bar_Scale) / Total_Emissions;   // bar length
    cout << Activities[i] << " [" << Activity_Emissions << " kg CO2]\t: ";
    for (int j = 0; j < Bar_Length; j++)  // Add the bar
    {
        cout << "#";
    }
    cout << "\n";
}
```

```
================================================
          Activity-wise Emissions Bar Chart
================================================
Car travel (km)          [25.83 kg CO2] : ########
Electricity usage (kWh)  [55.35 kg CO2] : ##################
Flights (km)             [15.99 kg CO2] : #####
Heating (hour)           [30.75 kg CO2] : ##########
Cooking (hour)           [18.45 kg CO2] : ######
```

➢ **Benefits:**

- ➢ **Clarity:** Users can easily identify activities with the highest emissions.
- ➢ **Engagement:** The visual output makes the data more interactive and user-friendly.

CHAPTER 5: TESTING

5.1 Test Plan

The program was tested against various scenarios to ensure robustness:

1. Valid inputs for all activities.
2. Invalid inputs such as non-numeric values and negative numbers.
3. Edge cases, including zero input and maximum possible input.

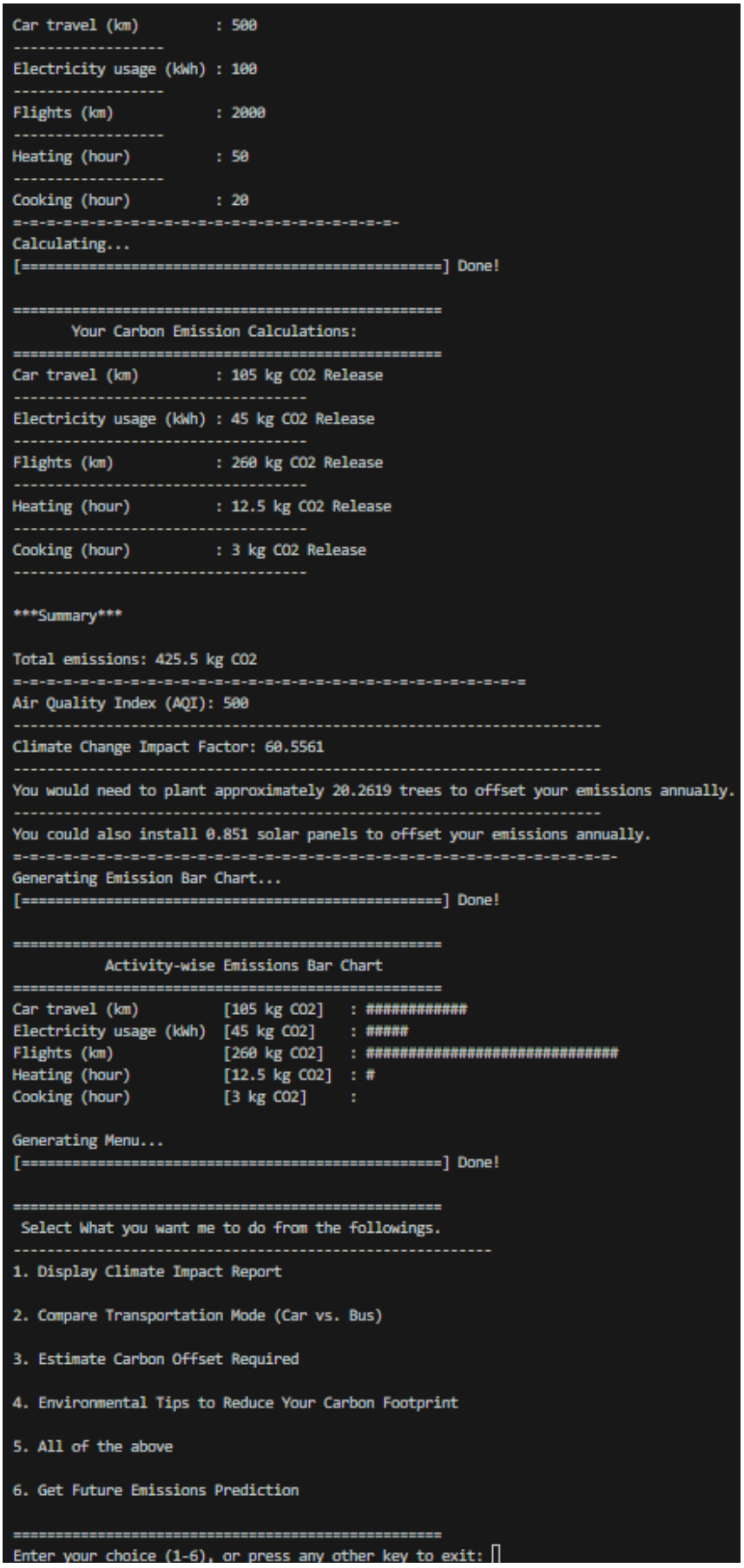
5.2 Test Cases

Test Case	Input	Expected Output	Result
Valid inputs	*Car: 100, Elec: 50*	*Correct emission calculations*	*Passed*
Invalid input (string)	*Car: abc*	*Error message: "Invalid input"*	*Passed*
Negative input	*Car: -10*	*Error message: "Input cannot be negative"*	*Passed*
Large emissions	*Car: 10000*	*AQI reaches max of 500*	*Passed*
Zero input	*All: 0*	*Total emissions = 0, AQI = 0*	*Passed*

Test Cast 1: Moderate Activity Level



Test Cast 2: High Activity Level



Test Cast 3: Low Activity Level



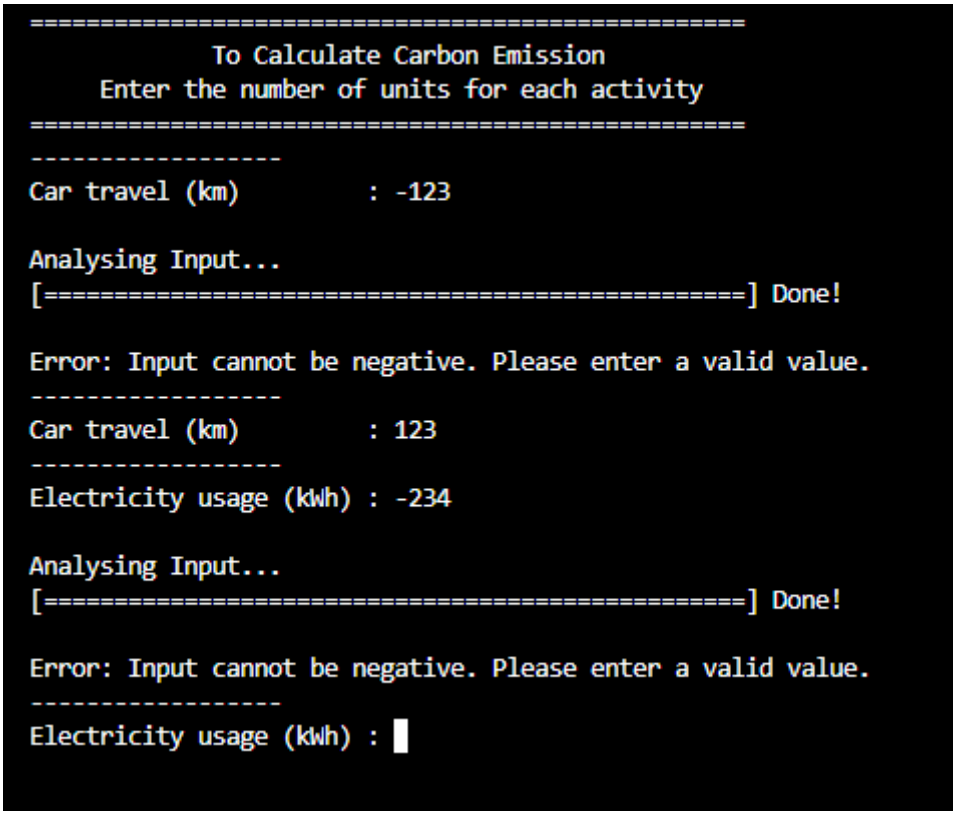
Test Cast 4: Zero Activity Level



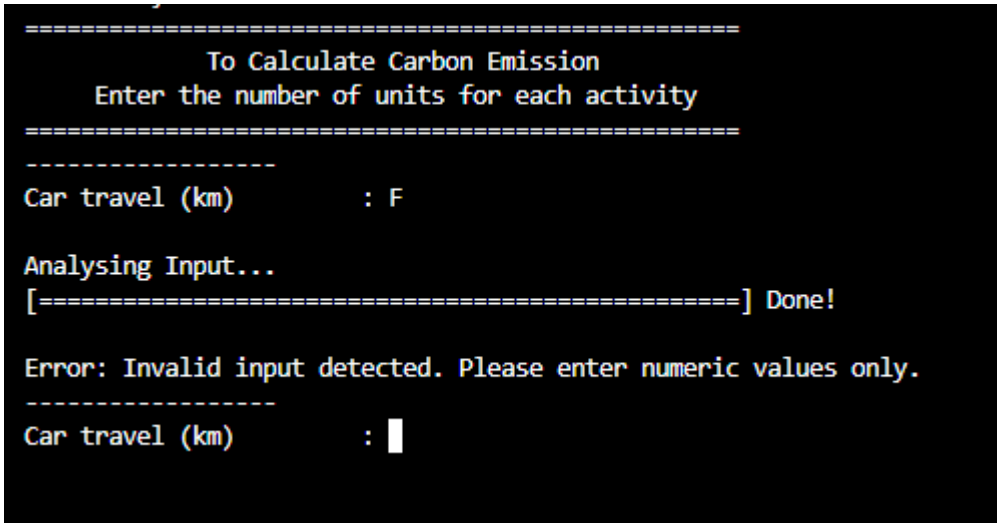
Test Cast 5: Extreme Activity Level



Test Cast 6: Negative Activity Level



Test Cast 7: Activity Level (Input character)



5.3 Results

All core functionalities performed as expected.

5.4 Error Handling

The program handles common errors like invalid input types and negative numbers efficiently. However, areas for improvement include:

1. Handling extremely large inputs that might cause overflow errors.
2. Providing more descriptive error messages for certain edge cases.

CHAPTER 6: USER INTERACTION FEATURES

The **Climate Change Impact Calculator** is designed with an interactive menu-driven interface, enabling users to explore various features seamlessly.

6.1 Menu Option

The program offers a wide range of options to suit different user needs, as displayed in the main menu:

```
cout << "\n=====================================================\n";
cout << " Select What you want me to do from the followings.\n";
cout << "-----------------------------------------------------\n";
cout << "1. Display Climate Impact Report\n\n";
cout << "2. Compare Transportation Mode (Car vs. Bus)\n\n";
cout << "3. Estimate Carbon Offset Required\n\n";
cout << "4. Environmental Tips to Reduce Your Carbon Footprint\n\n";
cout << "5. All of the above\n\n";
cout << "6. Get Future Emissions Prediction\n\n";
cout << "=====================================================\n";
cout << "Enter your choice (1-6), or press any other key to exit: ";
```

```
=====================================================
 Select What you want me to do from the followings.
-----------------------------------------------------
1. Display Climate Impact Report

2. Compare Transportation Mode (Car vs. Bus)

3. Estimate Carbon Offset Required

4. Environmental Tips to Reduce Your Carbon Footprint

5. All of the above

6. Get Future Emissions Prediction

=====================================================
Enter your choice (1-6), or press any other key to exit:
```

Each menu choice corresponds to a specific function:

1. Display Climate Impact Report

- Outputs total emissions, AQI, and climate impact factors.
- Provides activity-wise breakdown for user understanding:

```
cout << "\n=====================================\n"; // Displaying Climate Impact Report Activities wise
cout << "    --- Climate Impact Report ---\n";
cout << "=====================================\n";
cout << "      Activity-wise emissions\n";
cout << "    ----------------------------\n";
for (int i = 0; i < Total_Activities; ++i)
{
    double Activity_Emissions = User_Input[i] * CO2_Emission[i];
    cout << Activities[i] << ": " << Activity_Emissions << " kg CO2\n";
}
cout << "-----------------------------------\n";
cout << "Total emissions: " << Total_Emissions << " kg CO2\n";
cout << "-------------------------------------------------\n";
cout << "Air Quality Index (AQI): " << AQI << "\n";
cout << "-----------------------------------------------------\n";
cout << "Climate Change Impact Factor: " << Climate_Change_Impact << "\n";
cout << "=-=-=-
```

```
============================================================
            Transportation Mode Comparison
============================================================
Car travel emissions: 25.83 kg CO2
------------------------------------------------------------
Bus travel emissions (same distance): 6.15 kg CO2
------------------------------------------------------------
Switching to bus travel could save 19.68 kg CO2.
=-=-=-
```

```
==================================================
Environmental Tips to Reduce Your Carbon Footprint:
==================================================
1. Consider switching to electric vehicles or using public transport.
--------------------------------------------------------------------
2. Reduce energy consumption by using energy-efficient appliances.
--------------------------------------------------------------------
3. Opt for a plant-based diet to reduce emissions from food production.
--------------------------------------------------------------------
4. Install solar panels to reduce electricity-related emissions.
--------------------------------------------------------------------
5. Plant trees to offset your carbon emissions.
--------------------------------------------------------------------
```

5. Get Future Emissions Prediction

- Predicts future emissions based on current behavior:

```
double Future_Emissions = Total_Emissions * Years_Ahead;
cout << "In " << Years_Ahead << " years, your estimated total emissions will be: " << Future_Emissions << " kg CO2.\n";
```

```
Enter the number of years to predict your emissions
Enter Numeric Value Only: 20
------------------------------------------------------

Pridicting Future Emission please wait...
[==================================================] Done!
In 20 years, your estimated total emissions will be: 2927.4 kg CO2.
```

6. Process Bar

The Climate Change Impact Calculator includes a user-friendly, menu-driven interface. This menu allows users to navigate various features, such as viewing detailed climate reports, comparing transportation modes, and estimating carbon offsets.

To enhance user interaction, the program uses progress bars powered by the **<thread>** and **<chrono>** headers. These progress bars simulate real-time processing and provide feedback during operations like report generation, input validation, and file saving. For example, when a user enters invalid data, the program analyzes the input with a simulated progress bar, improving the overall user experience.

Usage of <thread> and <chrono>:

Progress Bars: The program simulates progress during lengthy tasks, such as calculating emissions or saving results to a file. This is achieved using std::this_thread::sleep_for to introduce delays.

```
#include <thread>      // For sleep functionality
```

```
for (int i = 0; i < 50; ++i)
{
    cout << "=" << flush;              // Print the progress bar incrementally
```

Time Delays: Time intervals are defined with std::chrono::milliseconds, ensuring precise control over the duration of each delay.

```
#include <chrono>      // For defining sleep duration
```

```
this_thread::sleep_for(chrono::milliseconds(20)); // Adjust speed with milliseconds
```

Advantages for User Interaction:

The progress bars provide immediate feedback, keeping users informed about the status of tasks.

Time delays make the program feel more interactive and realistic, mimicking actual data processing.

```
cout << "Calculating...\n[";
for (int i = 0; i < 50; ++i)
{
    cout << "=" << flush;              // Print the progress bar incrementally
    this_thread::sleep_for(chrono::milliseconds(30)); // Adjust speed with milliseconds
}
cout << "] Done!\n";
```

```
Calculating...
[==============================
```

```
Calculating...
[==================================================] Done!
```

6.2 File Saving

The program saves all calculated data and insights into a text file for future reference, ensuring users can revisit their results:

```
ofstream outputFile("ClimateImpactReport.txt");
```

```
outputFile << "Total emissions: " << Total_Emissions << " kg CO2\n";
outputFile << "-----------------------------------------------\n";
outputFile << "Air Quality Index (AQI): " << AQI << "\n";
outputFile << "-------------------------------------------------------\n";
```

```
outputFile.close();

    cout << "=-=-=-
```

The program saves the results of calculations, reports, and visualizations into a text file. To improve user engagement, a progress bar indicates the file-saving process. This progress bar is implemented using std::this_thread::sleep_for and std::chrono::milliseconds to simulate real-time operations.

```
cout << "Saving results to file...\n[";
for (int i = 0; i < 50; ++i)
{
    cout << "=" << flush;
    this_thread::sleep_for(chrono::milliseconds(20));
}
cout << "] Done!\n";
```

```
cout << "\nGenerating Climate Impact Report...\n[";
for (int i = 0; i < 50; ++i)
{
    cout << "=" << flush;              // Print the progress bar incrementally
    this_thread::sleep_for(chrono::milliseconds(20)); // Adjust speed with milliseconds
}
cout << "] Done!\n";
```

```
cout << "\nComparing Transportation Modes please wait...\n[";
for (int i = 0; i < 50; ++i)
{
    cout << "=" << flush;              // Print the progress bar incrementally
    this_thread::sleep_for(chrono::milliseconds(20)); // Adjust speed with milliseconds
}
cout << "] Done!\n";
```

```
cout << "\nPridicting Future Emission please wait...\n[";
for (int i = 0; i < 50; ++i)
{
    cout << "=" << flush;              // Print the progress bar incrementally
    this_thread::sleep_for(chrono::milliseconds(20)); // Adjust speed with milliseconds
}
cout << "] Done!\n";
```

CHAPTER 7: ENVIRONMENTAL INSIGHTS

This chapter explores the program's analytical and comparative capabilities, providing actionable insights into carbon emissions and sustainability.

7.1 Carbon Offset Calculation

The program calculates the number of trees or solar panels required to offset total emissions, making the concept of carbon neutrality more tangible:

```
double Trees_Required = Total_Emissions / 21.0; // 21 kg CO2 offset per tree annually
double SolarPanels_Required = Total_Emissions / 500.0; // 1 solar panel offsets 500 kg CO2/year
```

These metrics help users quantify the steps needed to counteract their carbon footprint.

7.2 Comparison Insights

The program compares emissions across different transportation modes to highlight sustainable choices:

```
double Car_Emission = User_Input[0] * CO2_Emission[0];
double Bus_Emission = User_Input[0] * 0.05; // Bus emissions: 0.05 kg CO2 per km (example value)
if (Car_Emission > Bus_Emission)
{
    cout << "Switching to bus travel could save " << (Car_Emission - Bus_Emission) << " kg CO2.\n";
}
```

This promotes informed decision-making, demonstrating the environmental benefits of public transport.

CHAPTER 8: CHALLENGES AND LIMITATIONS

8.1 Challenges

Here are some challenges which we faced during program implementation.

During the development of the Climate Change Impact Calculator, several technical and logical challenges were encountered. Below is a detailed account of the problems faced and the strategies used to overcome them:

1. **Handling Invalid and Negative Inputs**
    - **Problem:**
      During initial testing, invalid inputs such as strings or negative numbers caused the program to crash. For example, entering text like "abc" instead of numeric input generated an error, and entering negative values caused logical inconsistencies.
    - **Solution:**
      I implemented error-handling mechanisms using cin.fail() and conditional checks to validate input:

```
while (true)
  {
    cout << "-----------------\n";
    cout << Activities[i] << ": ";
    cin >> User_Input[i];

    if (cin.fail())         // Check for invalid input
    {
        cout << "\nAnalysing Input...\n[";
        for (int i = 0; i < 50; ++i)
        {
            cout << "=" << flush;              // Print the progress bar incrementally
            this_thread::sleep_for(chrono::milliseconds(20)); // Adjust speed with milliseconds
        }
        cout << "] Done!\n";
        cin.clear();        // Clear the error flags
        cin.ignore(100,'\n');   // Ignore invalid input upto 100 character or until new line
        cerr << "\n\aError: Invalid input detected. Please enter numeric values only.\n";
    }
    else if (User_Input[i] < 0)         // Check for negative input
    {
        cout << "\nAnalysing Input...\n[";
        for (int i = 0; i < 50; ++i)
        {
            cout << "=" << flush;              // Print the progress bar incrementally
            this_thread::sleep_for(chrono::milliseconds(20)); // Adjust speed with milliseconds
        }
        cout << "] Done!\n";
        cerr << "\n\aError: Input cannot be negative. Please enter a valid value.\n";
    }
    else
    {
        break;
    }
}
```

```
================================================
            To Calculate Carbon Emission
      Enter the number of units for each activity
================================================
-----------------
Car travel (km)         : Fifty

Analysing Input...
[================================================] Done!

Error: Invalid input detected. Please enter numeric values only.
-----------------
Car travel (km)         : -50

Analysing Input...
[================================================] Done!

Error: Input cannot be negative. Please enter a valid value.
-----------------
Car travel (km)         :
```

This ensured that the program only proceeded with valid, positive numeric inputs.

2. Scaling the Air Quality Index (AQI)

- **Problem:**
  Initially, the AQI calculation used a simple linear formula, which led to unrealistic results for higher emissions. For example, very high emissions sometimes produced AQI values exceeding 500, which contradicts the standardized scale.
- **Solution:**
  I revised the AQI calculation by introducing a tiered scaling system that adjusts the AQI based on emission ranges:

```
if (Total_Emissions <= 50)                                          //Standardized Scale:
{                                                                   // 0 to 50   : Good
    AQI = Total_Emissions * 10;  // Linear scaling for low emissions // 51 to 100 : Moderate
}                                                                   // 101 to 150: Unhealthy for sensitive groups
else if (Total_Emissions > 50 && Total_Emissions <= 250)             // 151 to 200: Unhealthy
{                                                                   // 201 to 300: Very Unhealthy
    AQI = 100 + (Total_Emissions - 50) * 2; // Moderate scaling     // 301 to 500: Hazardous
}
else if (Total_Emissions > 250 && Total_Emissions <= 1000)
{
    AQI = 300 + (Total_Emissions - 250) * 1.5; // Higher scaling
}
else
{
    AQI = 500;
}
```

This solution provided a more accurate and realistic representation of AQI.

3. Bar Chart Visualization

- **Problem:**
  The bar chart visualization initially lacked proportional scaling, which caused inconsistent and unreadable output for smaller emission values. For example, activities with minimal emissions would appear with no bars.
- **Solution:**
  I added a scaling factor (Bar_Scale) to ensure proportional bar lengths relative to total emissions:

```
int Bar_Length = (Activity_Emissions * Bar_Scale) / Total_Emissions;    // bar length

cout << Activities[i] << " [" << Activity_Emissions << " kg CO2]\t: ";

for (int j = 0; j < Bar_Length; j++)  // Add the bar
{
    cout << "#";
}
```

This adjustment ensured that every activity's contribution was visually represented, regardless of its size.

```
================================================
          Activity-wise Emissions Bar Chart
================================================
Car travel (km)          [25.83 kg CO2] : ########
Electricity usage (kWh)  [55.35 kg CO2] : ##################
Flights (km)             [15.99 kg CO2] : #####
Heating (hour)           [30.75 kg CO2] : ##########
Cooking (hour)           [18.45 kg CO2] : ######
```

4. File Handling for Report Saving

- **Problem:**
  During testing, the program sometimes failed to save reports to the file due to incorrect file handling or lack of permission.
- **Solution:**
  I implemented proper file handling techniques and added error checks to confirm successful file operations:

```
ofstream outputFile("ClimateImpactReport.txt");
if (outputFile.is_open())
{
outputFile << "-----------------------------------\n";
outputFile << "Total emissions: " << Total_Emissions << " kg CO2\n";
outputFile << "-------------------------------------------------\n";
outputFile << "Air Quality Index (AQI): " << AQI << "\n";
outputFile << "-----------------------------------------------------------\n";
outputFile << "Climate Change Impact Factor: " << Climate_Change_Impact << "\n";
outputFile << "-----------------------------------------------------------------------\n";
outputFile.close();

    cout << "=-=-=-
```

- **Solution:**
  I added comments and structured the code to allow future integration of user-defined values. Currently, the program uses:

```
double Bus_Emission = User_Input[0] * 0.05; // Bus emissions: 0.05 kg CO2 per km (example value)
```

In the future, this value can be replaced with user inputs or real-time data sources.

8.2 Limitations

Despite the solutions implemented, the program has some inherent limitations:

1. **Fixed Emission Factors**:
   The program uses predefined constants for emissions (e.g., 0.21 kg CO2/km for car travel). These values may not represent all scenarios, such as different car models or fuel types.
2. **No Real-Time Data Integration**:
   The program cannot fetch real-time data from sensors or external sources, such as live electricity usage or fuel consumption.
3. **Simplistic AQI Calculation**:
   Although the AQI scaling is tiered, it does not account for localized pollution factors or seasonal variations.
4. **User Dependency**:
   The accuracy of the program depends on precise and truthful user input, which may introduce errors.

By addressing these challenges and limitations, the program successfully delivers accurate calculations and actionable insights while setting the foundation for future improvements.

CHAPTER 9: CONCLUSION

9.1 Summary

The Climate Change Impact Calculator provides a comprehensive and interactive approach to understanding carbon emissions. From detailed activity breakdowns to actionable recommendations, the program empowers users to take measurable steps toward sustainability.

9.2 Future Work

To improve and expand the program's capabilities, the following enhancements are suggested:

1. **Integration with Real-Time Sensors**:
   Incorporate IoT devices to collect real-time data for electricity and fuel usage.
2. **Advanced Analytics**:
   Add features for lifecycle analysis and indirect emissions assessment.
3. **Mobile and Web Versions**:
   Develop a mobile or web-based interface for wider accessibility.
4. **Customizable Emission Factors**:
   Allow users to input specific values for vehicles, appliances, or local energy grids.

With these future enhancements, the Climate Change Impact Calculator aims to become an indispensable tool in the global fight against climate change, empowering individuals and organizations alike. These features not only enhance the user experience but also ensure the program's practicality as a tool for promoting environmental responsibility and informed decision-making.

CHAPTER 10: APPENDIX

10.1 Full Program Code

The complete program code is provided below for reference. It includes all modules, logic, and functionalities described in this report.

```cpp
#include <iostream>
#include <fstream>
#include <thread>      // For sleep functionality
#include <chrono>      // For defining sleep duration
#include <cmath>
using namespace std;

int main()
{
    const int Total_Activities = 5;      // Total Number of Activities

    string Activities[Total_Activities] = {"Car travel (km)\t\t", "Electricity usage (kWh)\t", "Flights (km)\t\t",
                                           "Heating (hour)\t\t", "Cooking (hour)\t\t"}; // Activities Names Predefined Array
    double CO2_Emission[Total_Activities] = {0.21, 0.45, 0.13, 0.25, 0.15};      // Carbon emission Predefined Array
    double User_Input[Total_Activities] = {0};

    cout << "================================================\n";
    cout << "            To Calculate Carbon Emission        \n";
    cout << "     Enter the number of units for each activity  \n";
    cout << "================================================\n";

    for (int i = 0; i < Total_Activities; i++)  // Input Activity data in Array
    {
        while (true)
        {
            cout << "------------------\n";
            cout << Activities[i] << ": ";
            cin >> User_Input[i];

            if (cin.fail())         // Check for invalid input
            {
                cout << "\nAnalysing Input...\n[";
                for (int i = 0; i < 50; ++i)
                {
                    cout << "=" << flush;              // Print the progress bar incrementally
                    this_thread::sleep_for(chrono::milliseconds(20)); // Adjust speed with milliseconds
                }
                cout << "] Done!\n";
                cin.clear();       // Clear the error flags
                cin.ignore(100,'\n');   // Ignore invalid input upto 100 character or until new line
                cerr << "\n\aError: Invalid input detected. Please enter numeric values only.\n";
            }
            else if (User_Input[i] < 0)        // Check for negative input
            {
                cout << "\nAnalysing Input...\n[";
                for (int i = 0; i < 50; ++i)
                {
                    cout << "=" << flush;              // Print the progress bar incrementally
```

```
                this_thread::sleep_for(chrono::milliseconds(20)); // Adjust speed with milliseconds
            }
            cout << "] Done!\n";
            cerr << "\n\aError: Input cannot be negative. Please enter a valid value.\n";
        }
        else
        {
            break;
        }
    }
}
cout<<"=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-\n";
double Total_Emissions = 0.0;
for (int i = 0; i < Total_Activities; i++)  // Calculating Total Carbon emissions
{
    Total_Emissions += User_Input[i] * CO2_Emission[i];
}
double AQI = 0;
if (Total_Emissions <= 50)                                          //Standardized Scale:
{                                                                   // 0 to 50   : Good
    AQI = Total_Emissions * 10;  // Linear scaling for low emissions // 51 to 100 : Moderate
}                                                                   // 101 to 150: Unhealthy for sensitive groups
else if (Total_Emissions > 50 && Total_Emissions <= 250)             // 151 to 200: Unhealthy
{                                                                   // 201 to 300: Very Unhealthy
    AQI = 100 + (Total_Emissions - 50) * 2; // Moderate scaling      // 301 to 500: Hazardous
}
else if (Total_Emissions > 250 && Total_Emissions <= 1000)
{
    AQI = 300 + (Total_Emissions - 250) * 1.5; // Higher scaling
}
else
{
    AQI = 500;
}
if (AQI > 500)
{
    AQI = 500;
}
double Climate_Change_Impact = log(Total_Emissions + 1) * 10; // Calculate Climate Change Impact Factor

double Trees_Required = Total_Emissions / 21.0; // 21 kg CO2 offset per tree annually
double SolarPanels_Required = Total_Emissions / 500.0; // 1 solar panel offsets 500 kg CO2/year

cout << "Calculating...\n[";
for (int i = 0; i < 50; ++i)
{
    cout << "=" << flush;             // Print the progress bar incrementally
    this_thread::sleep_for(chrono::milliseconds(30)); // Adjust speed with milliseconds
}
cout << "] Done!\n";

cout << "\n================================================\n";
cout << "       Your Carbon Emission Calculations:\n";
cout << "================================================\n";
for (int i = 0; i < Total_Activities; ++i)          // Activity-wise Emission Calculations
{
    double Activity_Emissions = User_Input[i] * CO2_Emission[i];
    cout << Activities[i] << ": " << Activity_Emissions << " kg CO2 Release\n";
    cout<<"---------------------------------\n";
}
cout << "\n***Summary***\n\n";
cout << "Total emissions: " << Total_Emissions << " kg CO2\n";
cout << "=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=\n";
cout << "Air Quality Index (AQI): " << AQI << "\n";
cout << "-------------------------------------------------------------------\n";
cout << "Climate Change Impact Factor: " << Climate_Change_Impact << "\n";
cout << "-------------------------------------------------------------------\n";
cout << "You would need to plant approximately " << Trees_Required << " trees to offset your emissions annually.\n";
cout << "-------------------------------------------------------------------\n";
cout << "You could also install " << SolarPanels_Required << " solar panels to offset your emissions annually.\n";
cout<<"=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=\n";

cout << "Generating Emission Bar Chart...\n[";
for (int i = 0; i < 50; ++i)
{
cout << "=" << flush;                          // Print the progress bar incrementally
this_thread::sleep_for(chrono::milliseconds(50)); // Adjust speed with milliseconds
}
cout << "] Done!\n";

cout << "\n================================================\n";     // Display Bar Chart for Activity-wise Emissions
cout << "           Activity-wise Emissions Bar Chart       \n";
cout << "================================================\n";

const int Bar_Scale = 50;  // Adjust the scale of the bar chart
for (int i = 0; i < Total_Activities; i++)
{
    double Activity_Emissions = User_Input[i] * CO2_Emission[i];
```

```
227         cout << "] Done!\n";
228         cout << "\n============================================================\n";
229         cout << "                Carbon Offset Suggestions                   \n";
230         cout << "============================================================\n";
231         cout << "1. You would need to plant approximately " << Trees_Required << " trees to offset your emissions annually.\n";
232         cout << "------------------------------------------------------------\n";
233         cout << "2. Install " << SolarPanels_Required << " solar panels to offset your emissions annually.\n";
234         cout << "------------------------------------------------------------\n";
235         cout << "3. Consider supporting renewable energy projects to reduce your footprint.\n";
236         cout << "------------------------------------------------------------\n";
237         cout << "4. Use energy efficient cooking methods such as induction cooking.\n";
238         cout<< " ------------------------------------------------------------\n";
239         cout << "5. Insulate your home to reduce heating needs.\n";
240         cout << "=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=\n";
241     }
242     if (choice == 4 || choice == 5)
243     {
244         cout << "\nGenerating Environmental Tips...\n[";
245         for (int i = 0; i < 50; ++i)
246         {
247             cout << "=" << flush;              // Print the progress bar incrementally
248             this_thread::sleep_for(chrono::milliseconds(20)); // Adjust speed with milliseconds
249         }
250         cout << "] Done!\n";
251         cout << "\n================================================\n";
252         cout << "Environmental Tips to Reduce Your Carbon Footprint:\n";   // Environmental Tips to reduce carbon footprint
253         cout << "================================================\n";
254         cout << "1. Consider switching to electric vehicles or using public transport.\n";
255         cout << "-----------------------------------------------------------------\n";
256         cout << "2. Reduce energy consumption by using energy-efficient appliances.\n";
257         cout << "-----------------------------------------------------------------\n";
258         cout << "3. Opt for a plant-based diet to reduce emissions from food production.\n";
259         cout << "-----------------------------------------------------------------\n";
260         cout << "4. Install solar panels to reduce electricity-related emissions.\n";
261         cout << "-----------------------------------------------------------------\n";
262         cout << "5. Plant trees to offset your carbon emissions.\n";
263         cout << "=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=\n";
264     }
265     if (choice == 6)        // Future Emissions Prediction
266     {
267         int Years_Ahead;     // Estimate future emissions (assuming no changes)
268         cout << "\nEnter the number of years to predict your emissions ";
269         cout << "\nEnter Numeric Value Only: ";
270         if (cin.fail() || Years_Ahead<0)
271         {
272             cin.clear();       // Clear the error flags
273             cin.ignore(1000,'\n');   // Ignore invalid input upto 1000 character or until new line
274             cerr << "\n\aError: Invalid input detected. Please enter numeric values only.\n";
275         }
276         else
277         {
278             break;
279         }
280         cout << "------------------------------------------------------------\n";
281         double Future_Emissions = Total_Emissions * Years_Ahead;
282         cout << "\nPridicting Future Emission please wait...\n[";
283         for (int i = 0; i < 50; ++i)
284         {
285             cout << "=" << flush;              // Print the progress bar incrementally
286             this_thread::sleep_for(chrono::milliseconds(20)); // Adjust speed with milliseconds
287         }
288         cout << "] Done!\n";
289         cout << "In " << Years_Ahead << " years, your estimated total emissions will be: " << Future_Emissions << " kg CO2.\n";
290     }
291     if (choice<1 || choice>6)
292     {
293         cout << "\nAnalyzing Choice...\n[";
294         for (int i = 0; i < 50; ++i)
295         {
296             cout << "=" << flush;              // Print the progress bar incrementally
297             this_thread::sleep_for(chrono::milliseconds(20)); // Adjust speed with milliseconds
298         }
299         cout << "] Done!\n\n";
300         cout << "Invalid Input Choice must be in the range of 1-6";
301     }
302     char Continue_Choice;
303     cout << "\nDo you want to make another choice? (y/n)";
304     cout << "\nPress any other key to exit: ";
305     cin >> Continue_Choice;
306 
307     if (Continue_Choice != 'y' && Continue_Choice != 'Y')
308     {
309         cout << "Exiting the program. Goodbye!" << endl;
310         break;
311     }
312 }
313 
314 
315 cout << "Saving results to file...\n[";
```

```
    cout << "Saving results to file...\n[";
    for (int i = 0; i < 50; ++i)
    {
        cout << "=" << flush;
        this_thread::sleep_for(chrono::milliseconds(20));
    }
    cout << "] Done!\n";
    ofstream outputFile("ClimateImpactReport.txt");

    if (outputFile.is_open())
    {
        outputFile << "\n=====================================\n"; // Displaying Climate Impact Report Activities wise
        outputFile << "    --- Climate Impact Report ---\n";
        outputFile << "=====================================\n";
        outputFile << "      Activity-wise emissions\n";
        outputFile << "    ---------------------------\n";
        for (int i = 0; i < Total_Activities; ++i)
        {
            double Activity_Emissions = User_Input[i] * CO2_Emission[i];
            outputFile << Activities[i] << ": " << Activity_Emissions << " kg CO2\n";
            outputFile << "-----------------\n";
        }
        outputFile << "-----------------------------------\n";
        outputFile << "Total emissions: " << Total_Emissions << " kg CO2\n";
        outputFile << "-----------------------------------------------\n";
        outputFile << "Air Quality Index (AQI): " << AQI << "\n";
        outputFile << "-----------------------------------------------------\n";
        outputFile << "Climate Change Impact Factor: " << Climate_Change_Impact << "\n";
        outputFile << "-------------------------------------------------------------------------\n";
        outputFile << "\n=================================================\n";
        outputFile << "           Activity-wise Emissions Bar Chart       \n";
        outputFile << "=================================================\n";

        for (int i = 0; i < Total_Activities; i++)
    {
        double Activity_Emissions = User_Input[i] * CO2_Emission[i];
        int Bar_Length = (Activity_Emissions * Bar_Scale) / Total_Emissions;
        outputFile << Activities[i] << " [" << Activity_Emissions << " kg CO2]: ";
        for (int j = 0; j < Bar_Length; j++)
        {
            outputFile << "#";
        }
        outputFile << "\n";
    }
    outputFile << "=================================================\n";

    if (choice == 2 || choice == 5)
    if (choice == 2 || choice == 5)
    {
        outputFile << "\n============================================================\n";
        outputFile << "                Transportation Mode Comparison              \n";
        outputFile << "============================================================\n";
        outputFile << "Car travel emissions: " << (User_Input[0] * CO2_Emission[0]) << " kg CO2\n";
        outputFile << "-------------------------------------------------------------------------\n";
        outputFile << "Bus travel emissions (same distance): " << (User_Input[0] * 0.05) << " kg CO2\n";
    }
    if (choice == 3 || choice == 5)
    {
        outputFile << "\n============================================================\n";
        outputFile <<  "                    Carbon Offset Suggestions             \n";
        outputFile << "============================================================\n";
        outputFile << "1. You would need to plant approximately " << Trees_Required << " trees to offset your emissions annually.\n";
        outputFile << "-----------------------------------------------------------\n";
        outputFile << "2. Install " << SolarPanels_Required << " solar panels to offset your emissions annually.\n";
        outputFile << "-----------------------------------------------------------\n";
        outputFile << "3. Consider supporting renewable energy projects to reduce your footprint.\n";
        outputFile << "-----------------------------------------------------------\n";
        outputFile << "4. Use energy efficient cooking methods such as induction cooking.\n";
        outputFile<< " -----------------------------------------------------------\n";
        outputFile << "5. Insulate your home to reduce heating needs.\n";
        outputFile<< " -----------------------------------------------------------\n";
    }
    if (choice == 4 || choice == 5)
    {
        outputFile << "\n=================================================\n";
        outputFile << "Environmental Tips to Reduce Your Carbon Footprint:\n";   // Environmental Tips to reduce carbon footprint
        outputFile << "=================================================\n";
        outputFile << "1. Consider switching to electric vehicles or using public transport.\n";
        outputFile << "-------------------------------------------------------------------------\n";
        outputFile << "2. Reduce energy consumption by using energy-efficient appliances.\n";
        outputFile << "-------------------------------------------------------------------------\n";
        outputFile << "3. Opt for a plant-based diet to reduce emissions from food production.\n";
        outputFile << "-------------------------------------------------------------------------\n";
        outputFile << "4. Install solar panels to reduce electricity-related emissions.\n";
        outputFile << "-------------------------------------------------------------------------\n";
        outputFile << "5. Plant trees to offset your carbon emissions.\n";
    }
    outputFile.close();

        cout << "=-=-=-
```

```
407     else
408     {
409         cout << "\aError: Unable to save results to file.\n";
410     }
411     return 0;
412 }
```

10.2 Sample Inputs and Outputs

This section provides example scenarios of user interactions with the program, showcasing its capabilities and output.

Sample Input:

```
================================================
            To Calculate Carbon Emission
     Enter the number of units for each activity
================================================
------------------
Car travel (km)         : 150
------------------
Electricity usage (kWh) : 140
------------------
Flights (km)            : 200
------------------
Heating (hour)          : 70
------------------
Cooking (hour)          : 90
```

Sample Output:

```
================================================
       Your Carbon Emission Calculations:
================================================
Car travel (km)         : 31.5 kg CO2 Release
-----------------------------------
Electricity usage (kWh) : 63 kg CO2 Release
-----------------------------------
Flights (km)            : 26 kg CO2 Release
-----------------------------------
Heating (hour)          : 17.5 kg CO2 Release
-----------------------------------
Cooking (hour)          : 13.5 kg CO2 Release
-----------------------------------

***Summary***

Total emissions: 151.5 kg CO2
=-=-=-
```

10.3 Formulas and Constants

1. CO2 Emissions Calculation

$$Emission = Activity\ Units \times \text{Emission Factor}$$

Example

For car travel:

$$\text{Emission} = 120\text{km} \times 0.21\text{kg CO2/km} = 25.2\text{kg CO2}$$

2. Air Quality Index (AQI)

The AQI is calculated using a tiered scaling system:

- $AQI = Emission \times \text{Scalling Factor}$

Scaling

$0 - 50: Good$
$51 - 100: Moderate$
$101 - 150: Unhealthy\ for\ sensitive\ groups$
$151 - 200: Unhealthy$
$201 - 300: Very\ Unhelthy$
$301 - 500: Hazardous$

3. Climate Change Impact Factor

$$Impact\ Factor = \log(Total\ Emission + 1) \times 10$$

4. Carbon Offset Requirements

- **Trees Required:**

$$Trees = \frac{Total\ Emission}{21.0}(kg\ CO2/tress\ annually)$$

- **Solar Panels Required:**

$$Panals = \frac{Total\ Emission}{500.0}(kg\ CO2/panal\ annually)$$

5. Emission Factors

Activity	Emission Factor (kg CO2/unit)
Car travel (km)	0.21

Activity	Emission Factor (kg CO2/unit)
Electricity usage (kWh)	0.45
Flights (km)	0.13
Heating (hour)	0.25
Cooking (hour)	0.15

ACKNOWLEDGMENTS

“We would like to express our gratitude to our project supervisor, Sir Ghulam Ali, for his guidance and support throughout this project. Additionally, we appreciate the collaborative efforts of our team members in making this project successful.”